MSIN0097 Predictive Analytics

Individual Coursework

Candidate Number: KZFD6

Word Count/Page Count: 1983/30

Table of Contents

1. Introduction and Project Scope

1.1 Problem Statement

Supply chain industries often face challenges such as fraudulent orders, resulting in significant financial losses. This project aims to tackle these issues by leveraging machine learning to detect patterns in order history, supplier reliability, and transaction anomalies. By applying predictive models to past records, we aim to reduce fraud risks and optimize resource allocation. To demonstrate this, we use a supply chain dataset (https://data.mendeley.com/datasets/8gx2fvg2k6/5) by Constante, Silva, & Pereira (2019). The model development was influenced by insights from a Kaggle project (https://www.kaggle.com/code/skloveyyp/comparison-of-classification-regression-rnn) by Shu (2021).

We will implement various traditional machine learning algorithms alongside a neural network model. Neural networks are well-suited for classification problems, as demonstrated by their effectiveness in credit scoring applications (Edmond & Girsang, 2020). We will evaluate the performance of neural networks against traditional machine learning models to determine which approach is more effective for fraud detection.

The full project is available on GitHub at https://github.com/rezabosnia/supply_chain_dataset.

1.2 Objectives

The primary objective of this project is to predict fraudulent orders. Key tasks include:

- Compare performances of traditional machine learning algorithms and neural network
- Choose the best model to predict fraudulent orders

The project workflow is shown in Figure 18.

2. Data Overview

The dataset consists of 180,519 rows and 53 columns.. Store locations are primarily in the American continent, particularly in the USCA market. The geographical distribution is visualized in Figure 2. Several columns contain missing values, particularly Customer Lname, Customer Zipcode, Order Zipcode, and Product Description. Due to excessive missing values, Order Zipcode and Product Description are dropped. Customer Zipcode null values are replaced with zero, while Customer Lname is merged with Customer Fname to form a full name column.

3. Exploratory Data Analysis

3.1 Correlation Heatmap

The correlation heatmap in Figure 3 reveals strong positive relationships between revenue-related metrics such as Sales, Sales per Customer, Order Item Total, and Product Price. Order Profit Per Order, Order Item Profit Ratio, and Benefit Order are also highly correlated. Notably, bulk orders might have lower unit prices, as indicated by a negative correlation between Order Item Quantity and Order Item Product Price. Late delivery risk is possibly linked to deviations between scheduled and actual shipping times. To avoid multicollinearity, we will later drop several highly correlated columns before proceeding with modeling.

3.2 Timeseries Analysis

Figure 6 shows the line graphs for time series analysis. Overall, the sales fluctuate within the period of dataset, before they rise at the start of 2017 and decreased after Q3 2017. In a weekly basis, Tuesday has the lowest number of sales, and the number rises throughout the week until it peaks on Friday. This shows that orders might increase as the week approaches the weekend. In hourly basis, the lowest sales are around 6 pm when people might commute at that time, and the highest peak is at 3 pm when people usually have afternoon breaks. In monthly basis, the sales peaks October. This makes sense because most of the big holidays come after October such as Halloween, Christmas, and New Year.

3.3 Loss, Fraud, and Late Delivery Analysis

The highest product losses are recorded for Cleats, Men's Footwear, and Women's Apparel which are also among top fraud prone products. While Western Europe, Central America, and South America lead in total sales, they also experience the most significant losses and have the highest number of late deliveries. Among individual customers, Mary Smith leads with 528 fraudulent orders, followed by Robert Smith with only 28. Loss per product and region are shown in Figure 8, fraud-prone products is shown in Figure 9, and fraud customers are shown in Figure 10.

3.4 RFM Analysis and Customer Segmentation

We will inspect Recency, Frequency, and Monetary Value (RFM) variables to analyze customer's behaviors. We aggregate the orders by Order Customer ID. The count of orders will be the frequency, the difference between maximum order date and the individual order date is the recency, while monetary value is in a new column called TotalPrice, which is Order Item Quantity times Order Item Total. Figure 14 shows that all those three data are skewed. We transform the RFM variables into quantiles before clustering them using KMeans clustering. The Elbow method shows that the number of optimal clusters is 4, and we can see the 3D scatterplot visualization result of the clustering on Figure 15.

Based on the scatterplot, we can define the clusters into four groups. At-Risk Customers have low recency scores, indicating they are likely to churn, making discounts and promotions effective retention strategies. New Customers have high recency scores, low frequency, and low monetary

value, suggesting they are recent buyers with potential for growth by targeted promotions. High-Value Customers rank high across all three RFM metrics, making them ideal candidates for upselling, loyalty programs, and product diversification. Finally, Regular Customers exhibit moderate scores across all variables, meaning while they could benefit from promotional efforts, they are not the highest priority for intervention.

3.5 Feature Engineering

Before proceeding with machine learning analysis, feature engineering is performed to enhance model performance. The customer segmentation results are merged into the main dataset based on the Order Customer ID. Additionally, a binary "late_delivery" column is created to summarize columns related to late delivery information. Among the 180,519 orders, 98,977 are late deliveries, accounting for 54.83% of total orders. Finally, a binary "fraud" column is created, derived from Order Status columns, for classification purposes. Among the 180,519 orders, 4,062 are suspected fraud cases, accounting for 2.25% of total orders.

4. Modelling

4.1 Prepare Training Data

First, we decided to drop several columns that might give redundant information such as Delivery Status, Late Delivery Risk, Order Status, Order_month_year, and order date (DateOrders). Secondly, we will encode the categorical variables using LabelEncoder library. Following this, we split the dataset into train data and test data using train_test_split with 0.2 test size. We apply StandardScaler to standardize the features. The dataset is now ready for modeling.

4.2 Fraud Prediction using Traditional ML Models

We predict fraudulent orders using 6 traditional machine learning models, which are KNN, Decision Tree, Random Forest, Logistic Regression, Extra Trees, and Extreme Gradient Boosting. The summary of the evaluation results is shown in Table 1. Based on the summary, XGBoost is the best model, having the highest overall score. Furthermore, we use cross-validation techniques on XGBoost model to show that the average accuracy of the model is 96% +/- 4% depending on the training subset. However, the XGBoost has low recall and f1 score, which shows that the model struggles to identify fraudulent orders correctly. We've done hyperparameter tuning using Random Search method to further improve the model performances. Although it increases performance on detecting frauds, it made the training accuracy score increase to 100%, which might indicate overfitting.

4.3 Fraud Prediction using Neural Network

A neural network model is also trained to predict fraud. The neural network outperforms XGBoost across all key metrics, including test accuracy, recall, precision, and F1 score. Notably, its training accuracy is 97.8%, which indicates that the model is not overfitting and generalizes well to unseen data. This suggests that the neural network is more effective than traditional machine learning

models in detecting fraudulent transactions in this dataset. The detailed architecture of the neural network is provided in Table 2.

4.4 Dimension Reduction

The original models use 45 variables, which may cause scalability and computational challenges. To address this, we reduce the number of features while preserving critical information. Figure 16 demonstrates that 20 variables explain 86.67% of the variance. Additionally, we apply feature importance ranking using the Random Forest algorithm as an alternative feature selection method. Figure 17 highlights the top 20 influential features, with Order City, Category ID, Customer Segment, Department Name, and Department ID ranking among the most significant.

Table 1 shows that while accuracy remains stable, recall, precision, and F1-score fluctuate, particularly in traditional models. Dimensionality reduction significantly impacts tree-based models and logistic regression because they rely on original feature values. PCA transforms features into new principal components that do not directly correspond to the original features. while feature selection removes certain attributes or certain interactions between multiple features, affecting the model performance. In contrast, neural networks are more resilient, as they learn feature representations internally, making them less sensitive to feature transformations and maintaining stable performance. Reducing the number of features in neural networks also enhances scalability and computational efficiency, allowing the model to train faster while preserving predictive power.

4.5 Handling Overfitting

Most of the models performed exceptionally well, achieving accuracy above 97%. While this appears promising, it could indicate overfitting. We tackle this issue by solving data imbalance and fine-tune the hyperparameters of the neural network.

The labeled fraud data accounts for only 2.25% of the total dataset. This imbalance raises the possibility that the models are achieving high accuracy simply by predicting the non-fraudulent transactions. To mitigate this, we employ SMOTE (Synthetic Minority Over-sampling Technique) to up sample the fraud cases.

We experimented with different sampling strategies (0.2, 0.3, 0.4, 0.5, and auto) and found that the auto strategy, which up samples the minority class to match the majority class, yielded the best results, reaching an accuracy of 99.5%.

Additionally, we tested two versions of the neural network after solving data imbalance, one using 45 features and another with a reduced set of 20 features. The model using all 45 features has higher accuracy than the reduced feature set. However, this still contains overfitting risk, as overfitting occurs when a model is too complex relative to the amount and noisiness of the training data, leading to poor generalization on unseen data (Géron, 2022). Therefore, the 20-feature model is preferable as it reduces the risk of overfitting while maintaining strong

performance on other evaluation metrics. Additionally, a simpler model requires less computational power, making it more efficient for real-world deployment.

We further improve the neural network by tuning several hyperparameter, but it actually decreases the accuracy of the model severely. We then apply regularization techniques such as L2 and dropout to improve the model. The final neural network infrastructure is shown on Table 3. The final neural network uses 20 up sampled features to balance the dataset, incorporating regularization methods and comparing its performance to the previous model without regularization. After applying regularization, the model demonstrates better generalization while maintaining strong performance across key metrics. The training accuracy decreased slightly (98.3% → 97.7%), indicating reduced overfitting, meaning the model is no longer memorizing the training data excessively. However, this decrease comes with improvements in test accuracy (96.2% → 97.8%), recall (96.2% → 97.8%), and precision (96.7% → 97.8%), leading to an overall higher F1 score (96.4% → 96.6%). The increased recall suggests that the model is detecting more fraud cases correctly, while higher precision means it is making fewer false positive classifications. Since fraud detection prioritizes recall to minimize false negatives while also maintaining strong precision, the regularized model achieves a much better balance. This confirms that applying infrastructure regularization alongside oversampling improved fraud detection performance and reduced the risk of overfitting, making this the preferred model.

5. Conclusion

This project successfully developed a machine learning-based fraud detection system for supply chain operations, leveraging both traditional models and neural networks to identify fraudulent orders. Among traditional models, XGBoost achieved the best performance with 96% accuracy, but its low recall and F1-score indicated difficulty in detecting fraudulent transactions. In contrast, the neural network model outperformed traditional models on overall evaluation metrics.

We found that dimensionality reduction significantly impacted traditional models but had minimal effect on neural networks, which remained robust while improving scalability and computational efficiency. To address overfitting, we upsampled fraud orders and applied regularization, leading to better generalization and improved fraud detection accuracy.

In conclusion, this project demonstrates that in this supply chain dataset, we can use machine learning for fraud detection, and neural network performs better than traditional machine learning models.

REFERENCES

Aurélien Géron (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. ‘O’Reilly Media, Inc.’

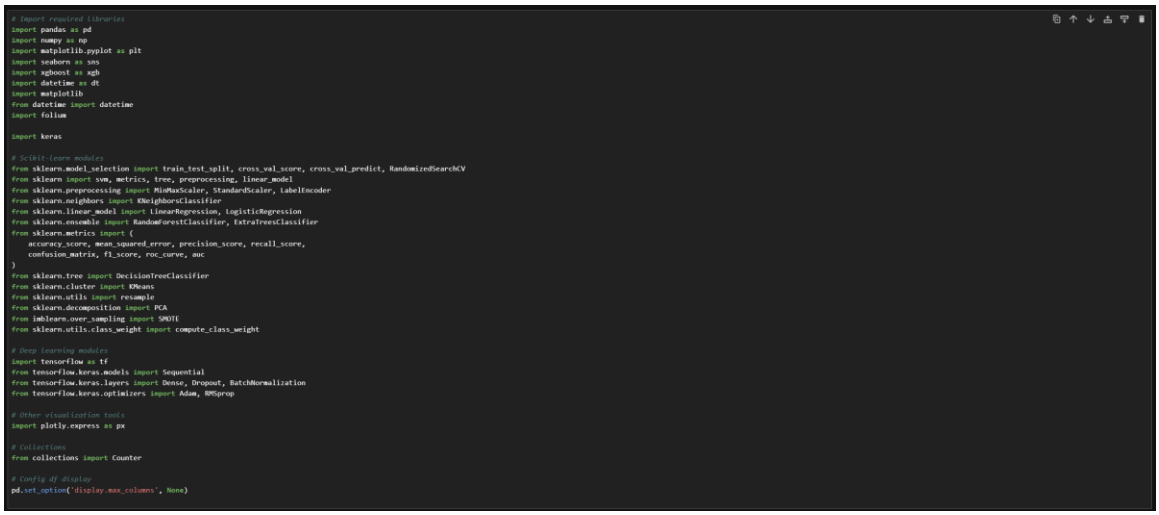
Constante, F., Silva, F. and Pereira, A. (2019). DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS. data.mendeley.com, [online] 5. doi:https://doi.org/10.17632/8gx2fvg2k6.5.

Edmond, C. (2020). Classification Performance for Credit Scoring using Neural Network. International Journal of Emerging Trends in Engineering Research, 8(5), pp.1592–1599. doi:https://doi.org/10.30534/ijeter/2020/19852020.

Kai Shu (2021). Comparison of Classification, Regression & RNN. [online] Kaggle.com. Available at: https://www.kaggle.com/code/skloveyyp/comparison-of-classification-regression-rnn [Accessed 6 Mar. 2025].

APPENDICES

APPENDIX 1: Figure 1



```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import xgboost as xgb
import datetime as dt
import matplotlib
from datetime import datetime
import folium

import keras

# Scikit-learn modules
from sklearn.model_selection import train_test_split, cross_val_score, cross_val_predict, RandomizedSearchCV
from sklearn import svm, metrics, tree, preprocessing, linear_model
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.metrics import (
    accuracy_score, mean_squared_error, precision_score, recall_score,
    confusion_matrix, f1_score, roc_curve, auc
)
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
from sklearn.utils import resample
from sklearn.decomposition import PCA
from imblearn.over_sampling import SMOTE
from sklearn.utils.class_weight import compute_class_weight

# Deep learning modules
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam, RMSprop

# Other visualization tools
import plotly.express as px

# Collections
from collections import Counter

# Config df display
pd.set_option('display.max_columns', None)
```

Figure 1 Libraries List

APPENDIX 2: Figure 2

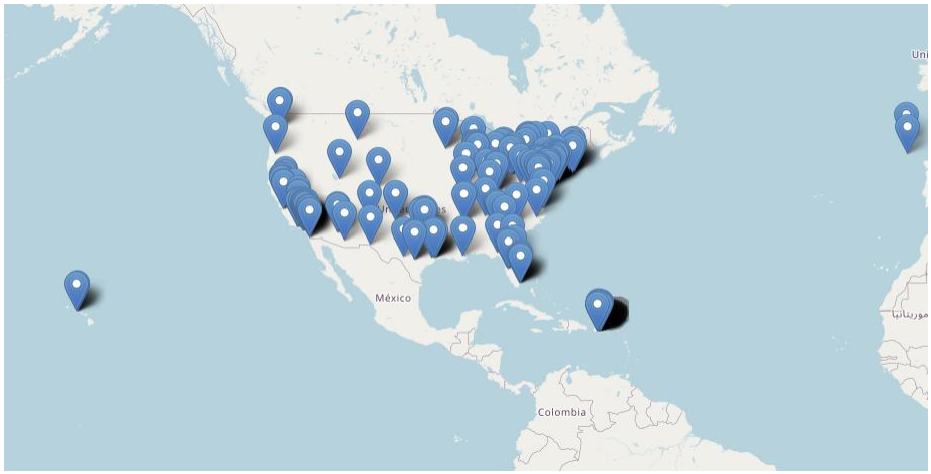


Figure 2 Folium Map of Store Locations

APPENDIX 3: Figure 3

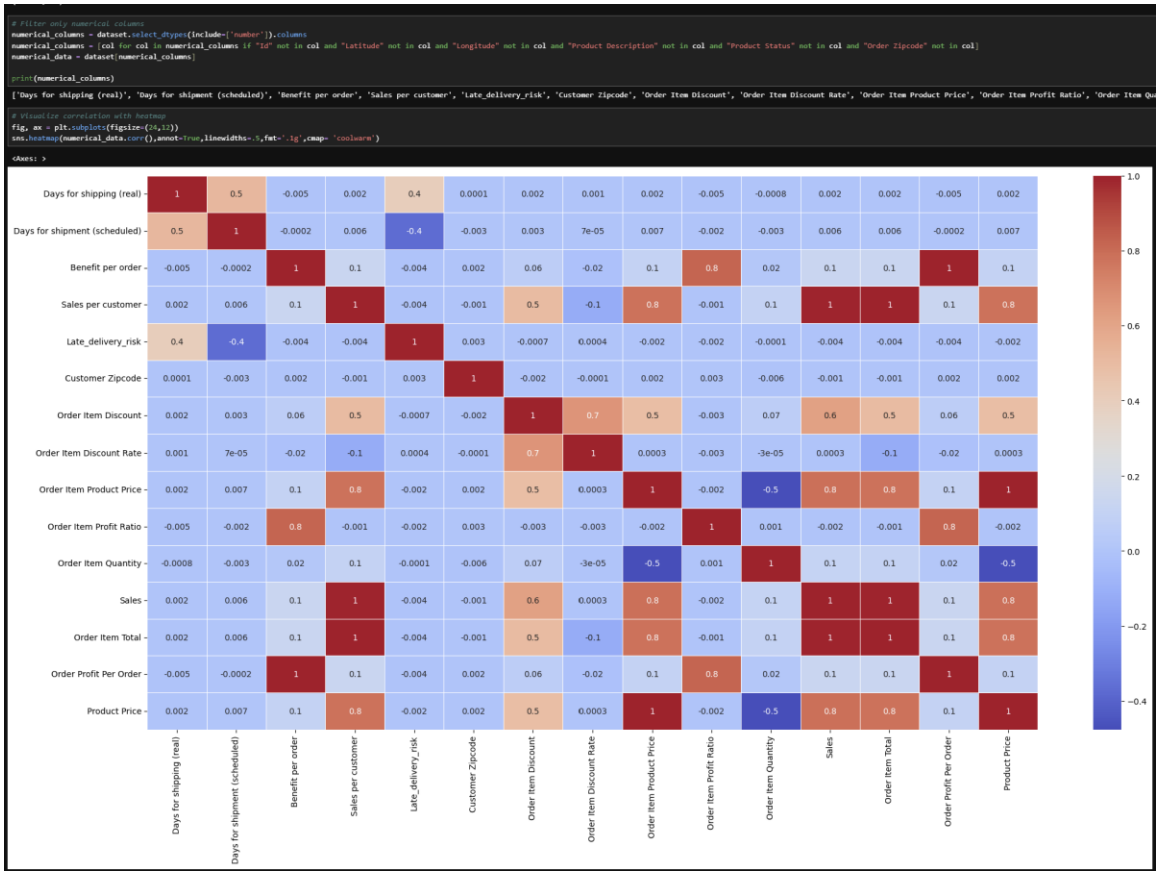
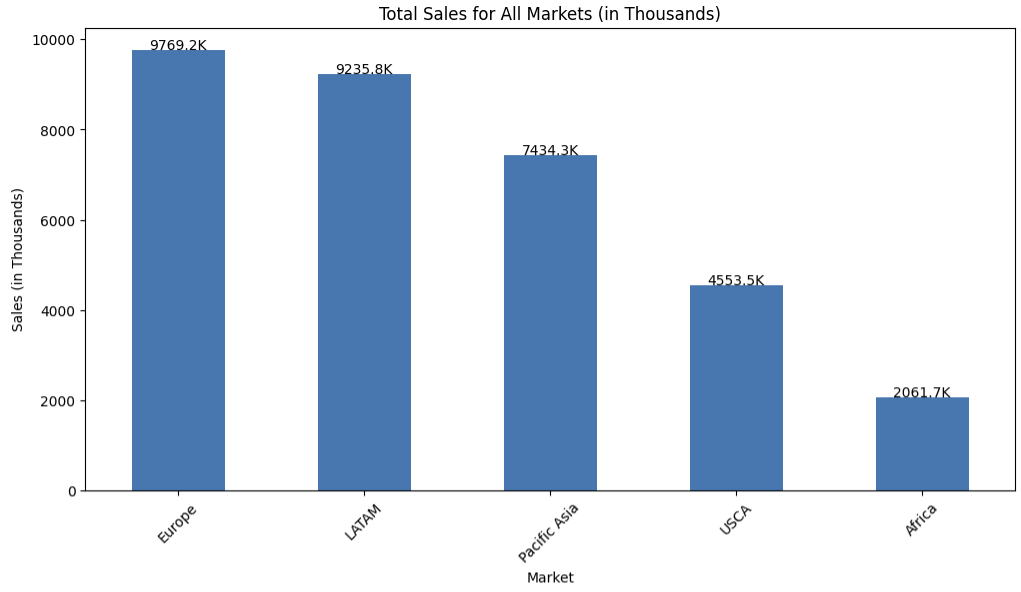


Figure 3 Correlation Heatmap

APPENDIX 4: Figure 4



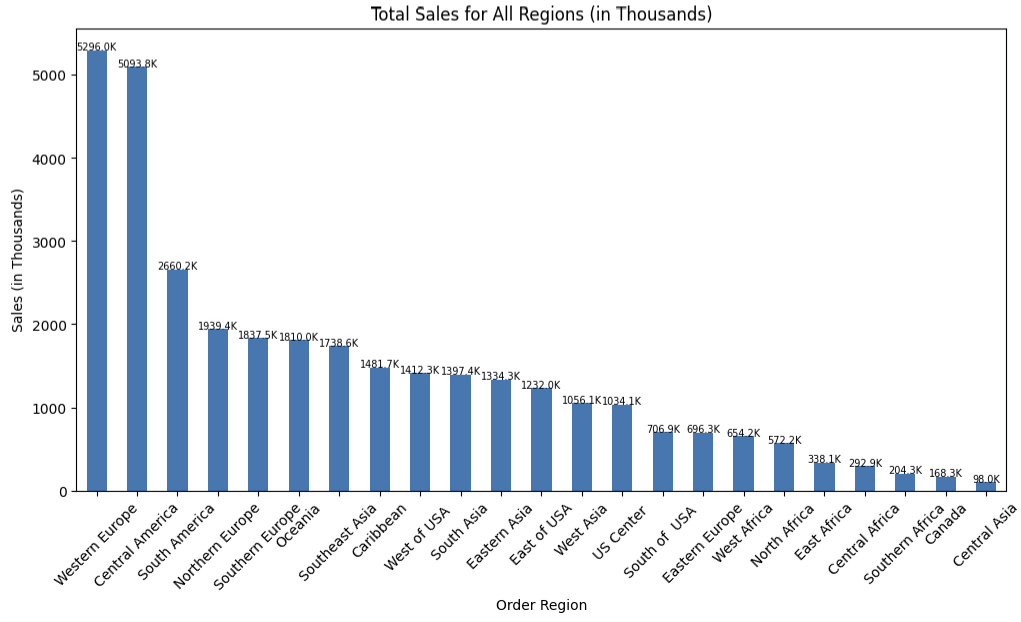


Figure 4 Total Sales Distribution based on Market and Region

APPENDIX 5: Figure 5

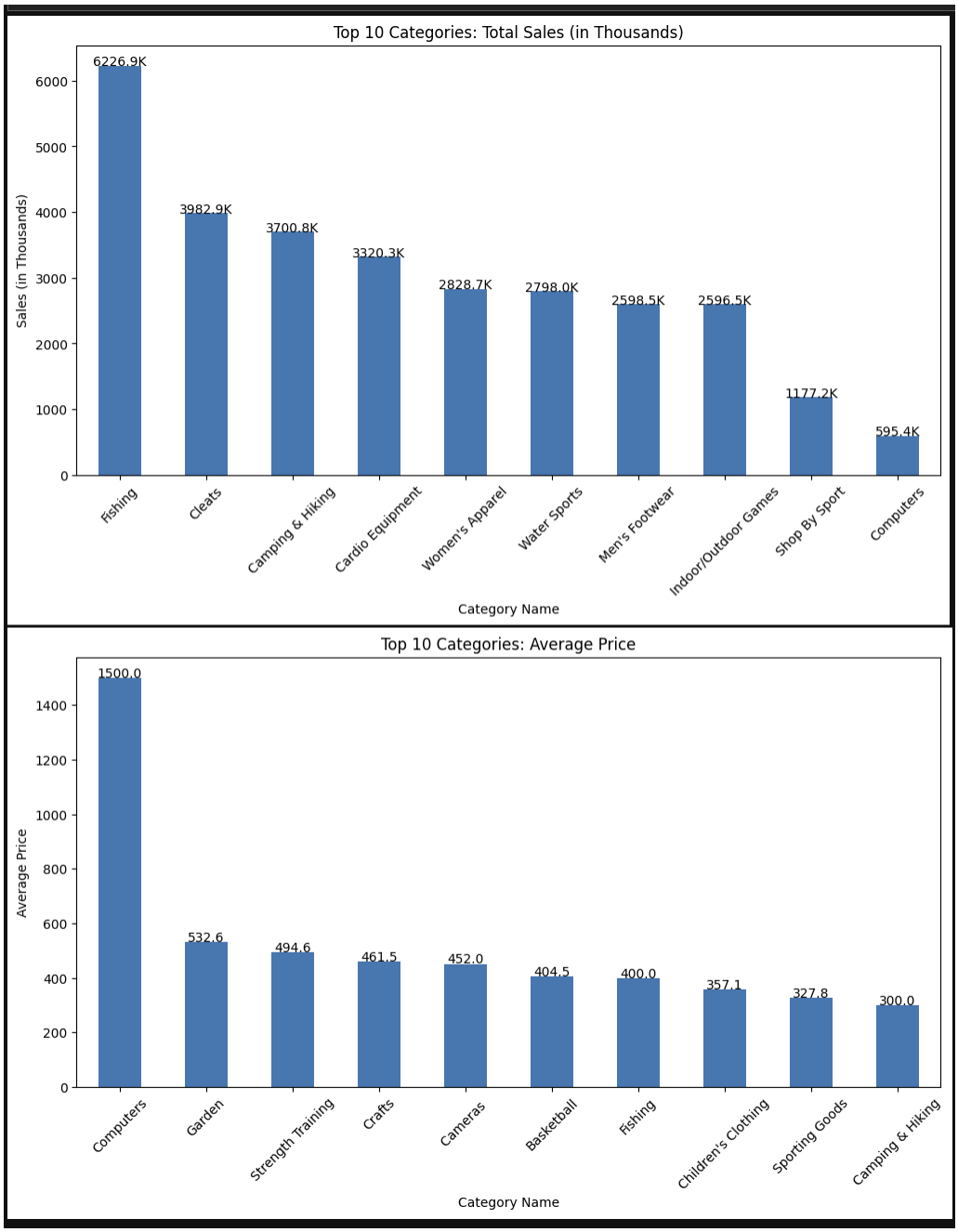


Figure 5 Total Sales per Category

APPENDIX 6: Figure 6

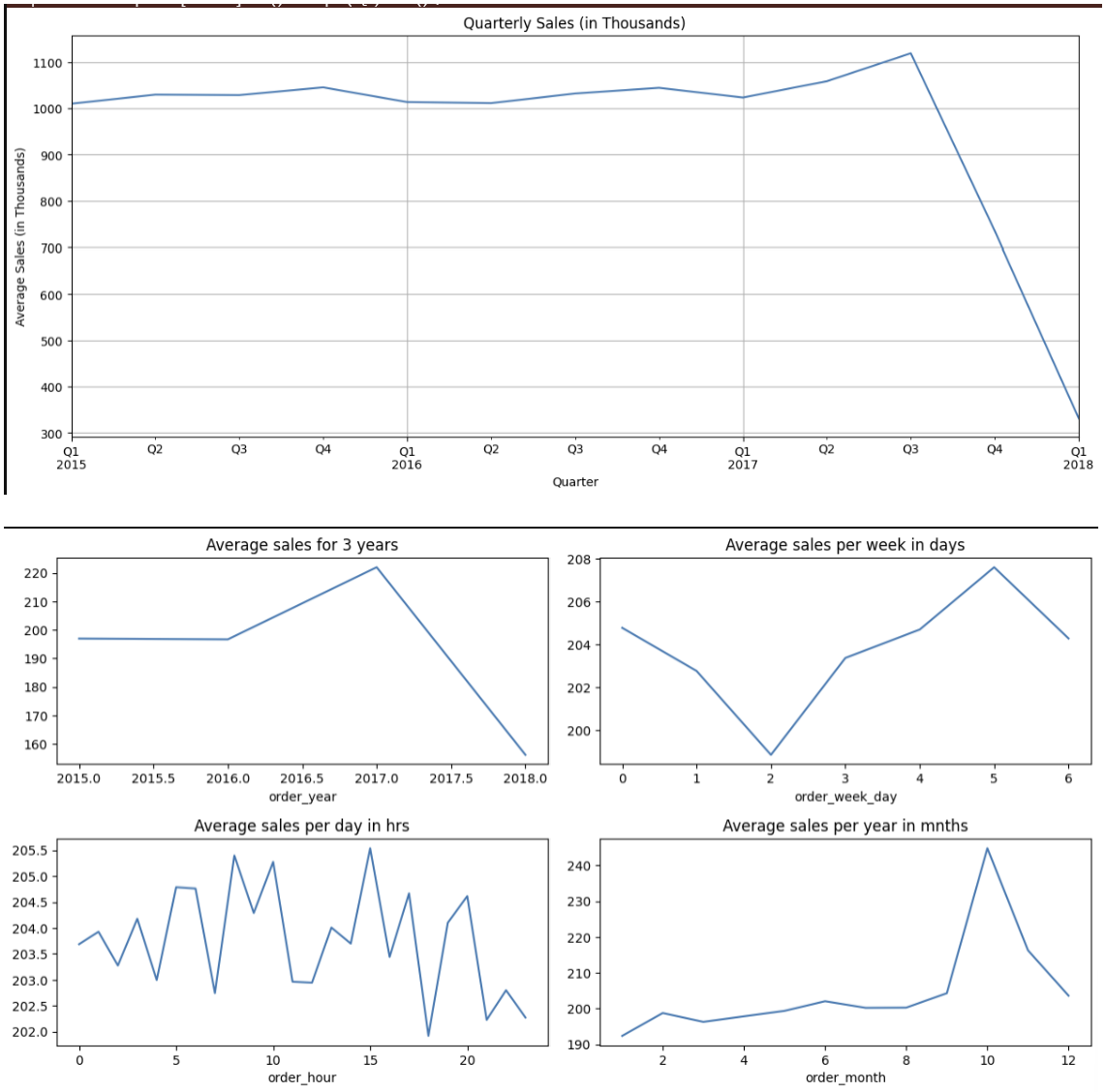
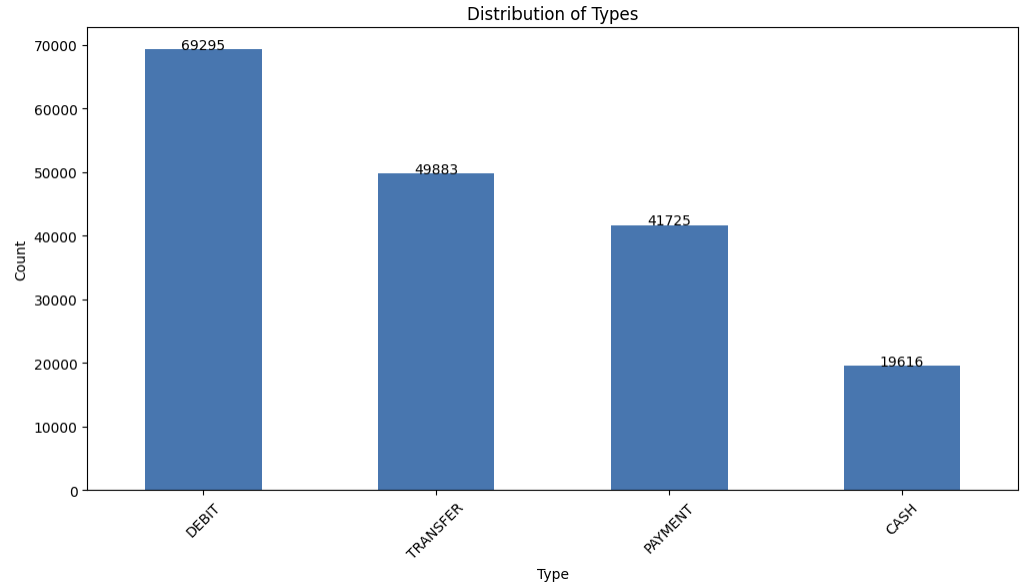


Figure 6 Timeseries Linegraph

APPENDIX 7: Figure 7



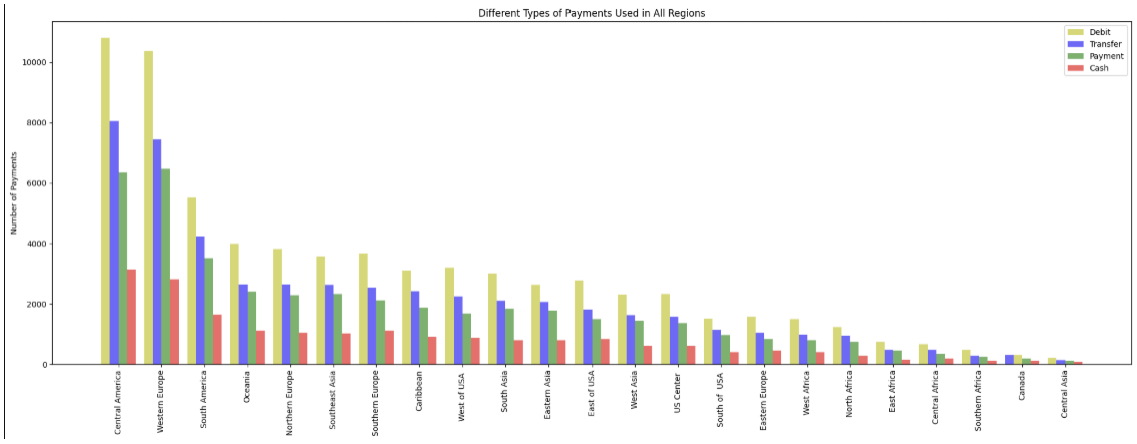


Figure 7 Payment Type Distribution across Regions

APPENDIX 8: Figure 8

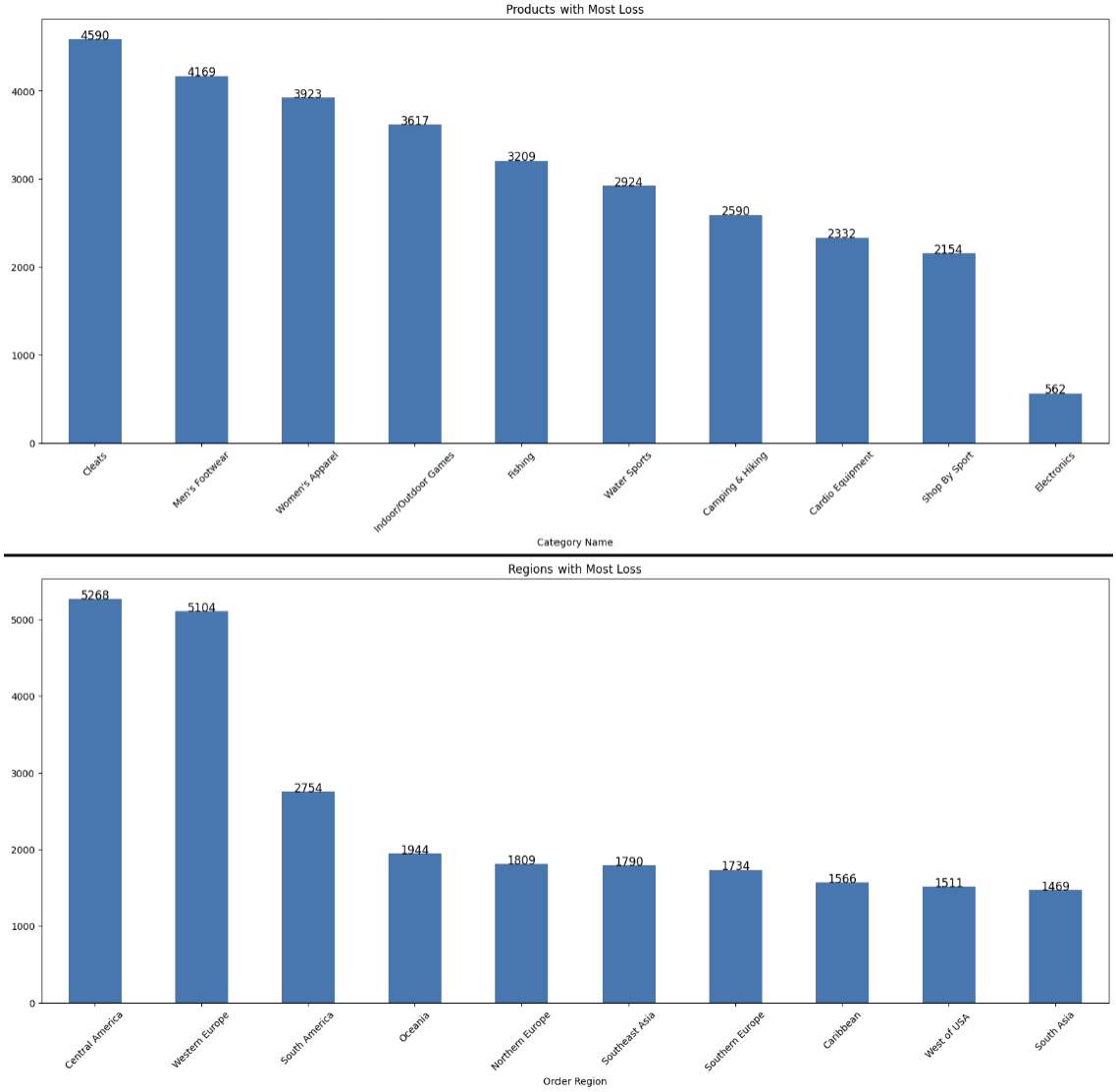


Figure 8 Loss per Products and Regions

APPENDIX 9: Figure 9

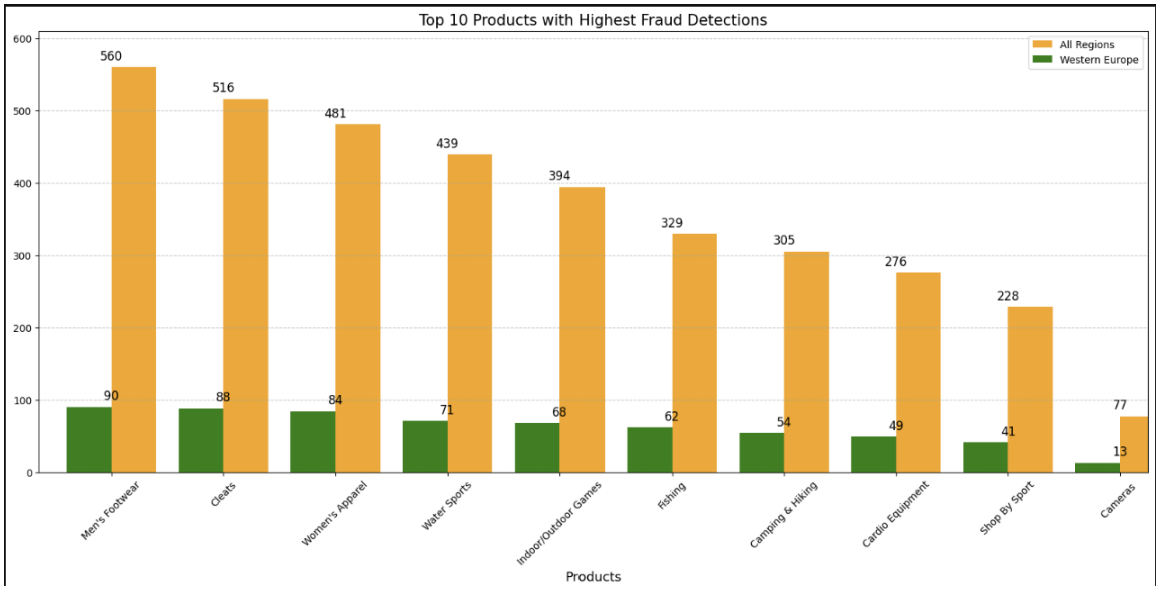


Figure 9 Product with Highest Fraud Detection

APPENDIX 10: Figure 10

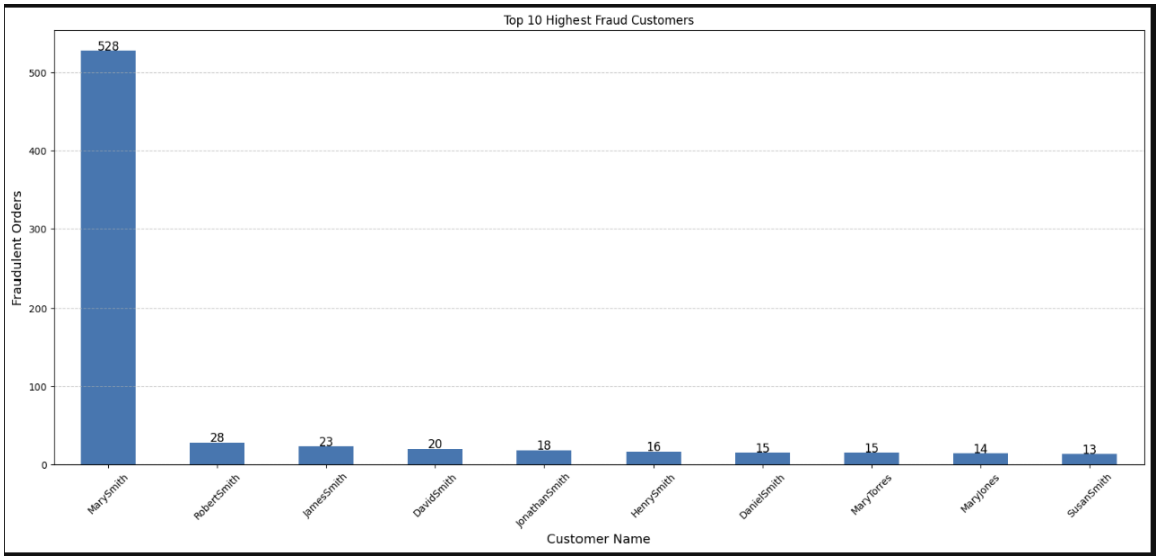


Figure 10 Top 10 Highest Fraud Customers

APPENDIX 11: Figure 11

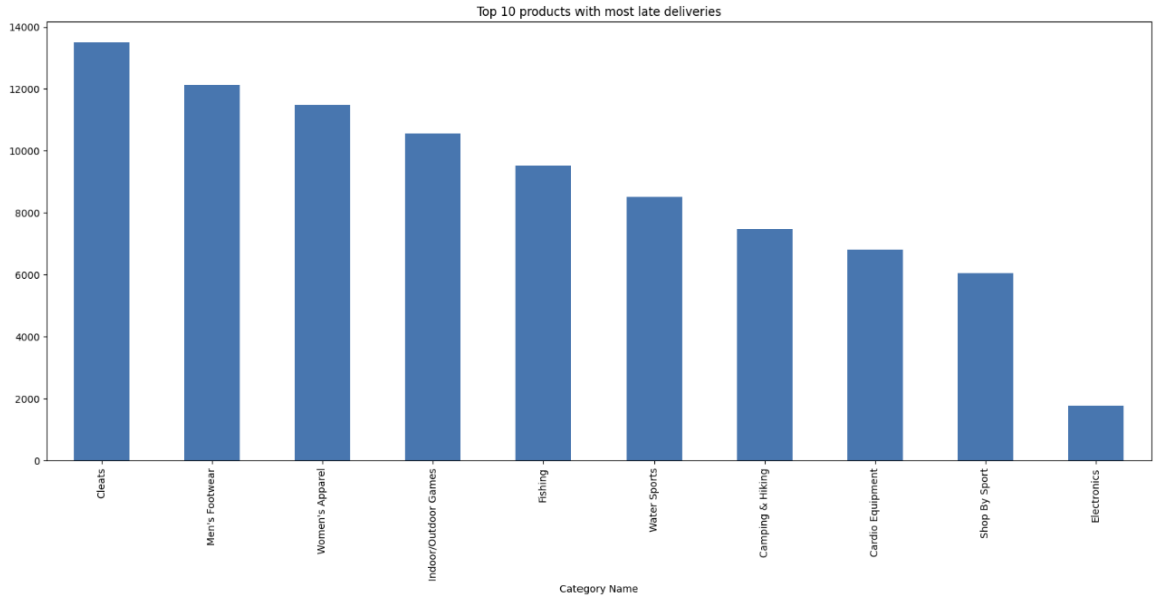


Figure 11 Products with Most Late Deliveries

APPENDIX 12: Figure 12

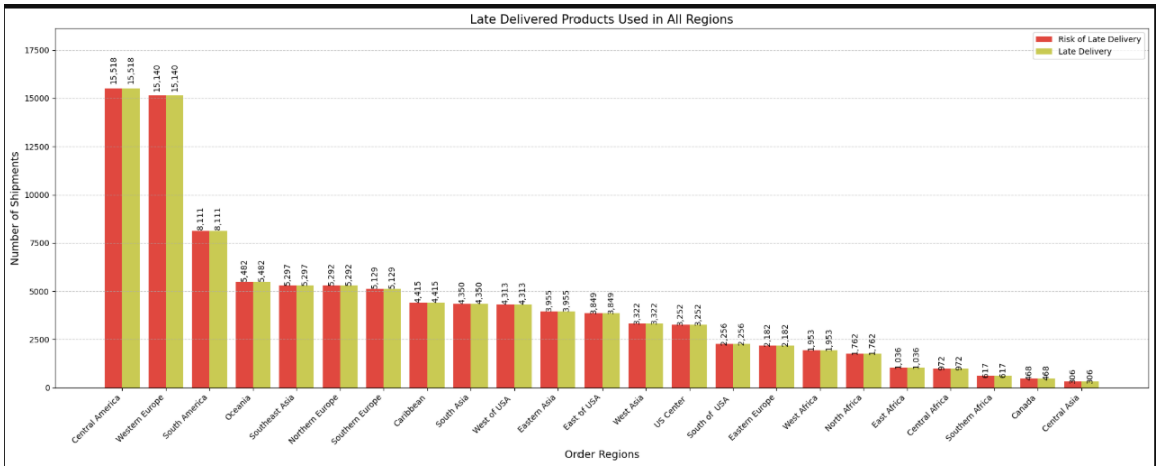


Figure 12 Amount of Late Delivered Products per Region

APPENDIX 13: Figure 13

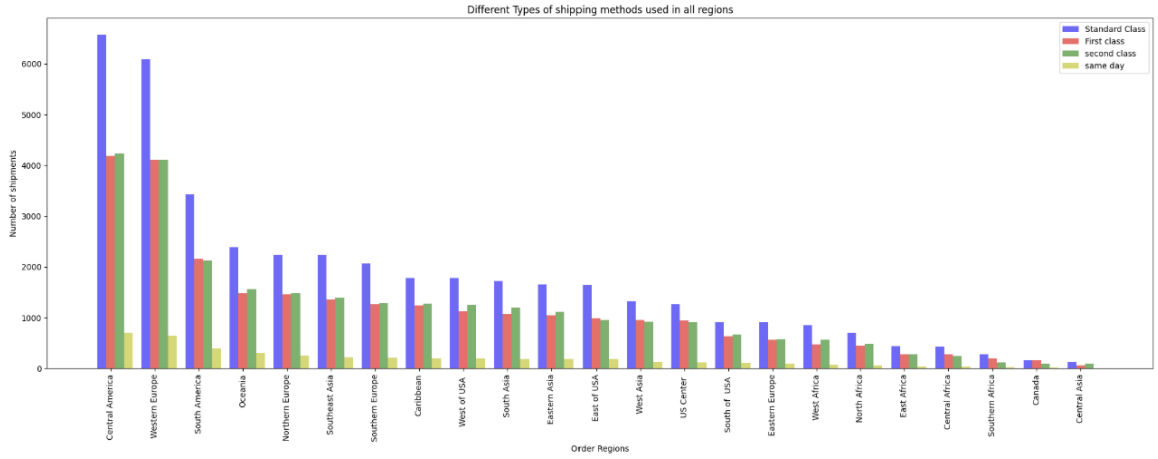


Figure 13 Shipping Methods of Late Deliveries in All Regions

APPENDIX 14: Figure 14

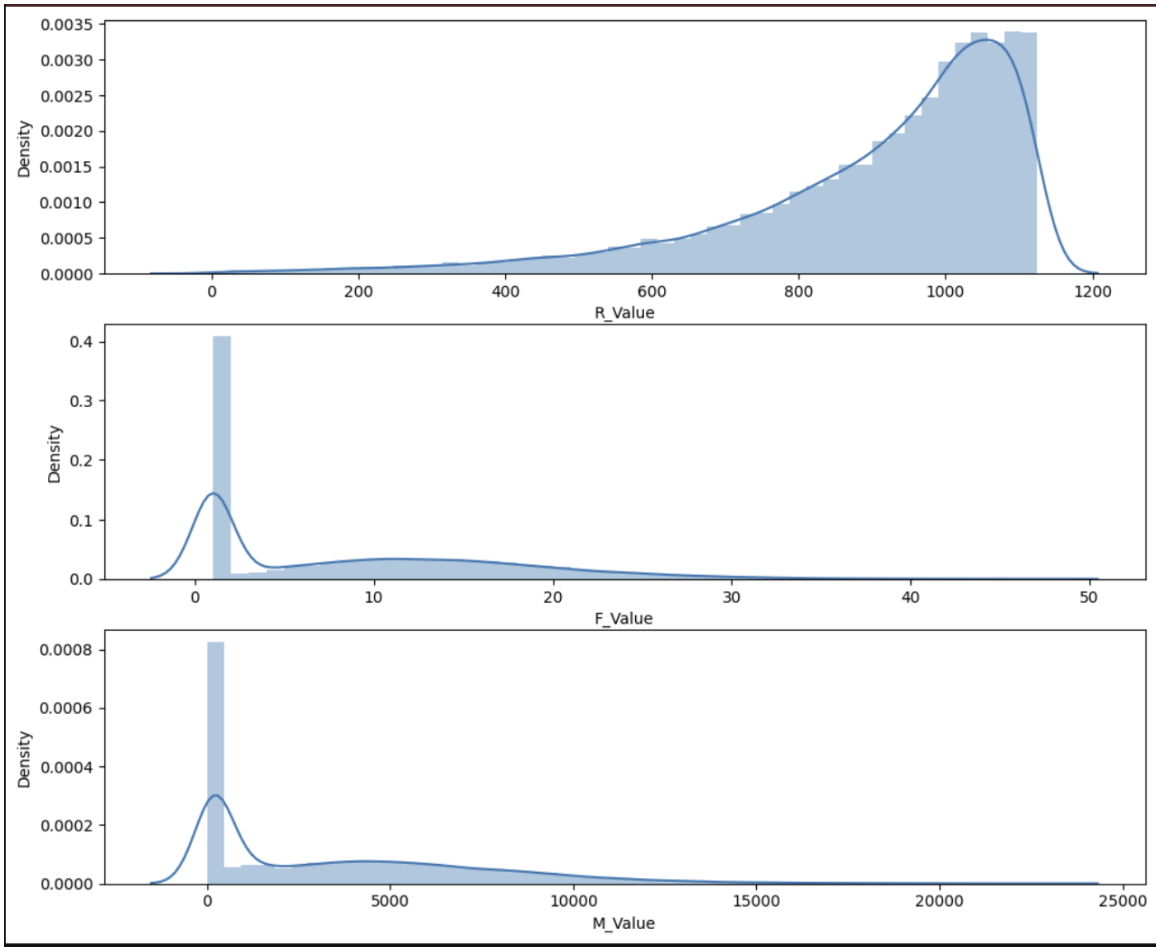


Figure 14 Distribution of RFM Variables

APPENDIX 15: Figure 15

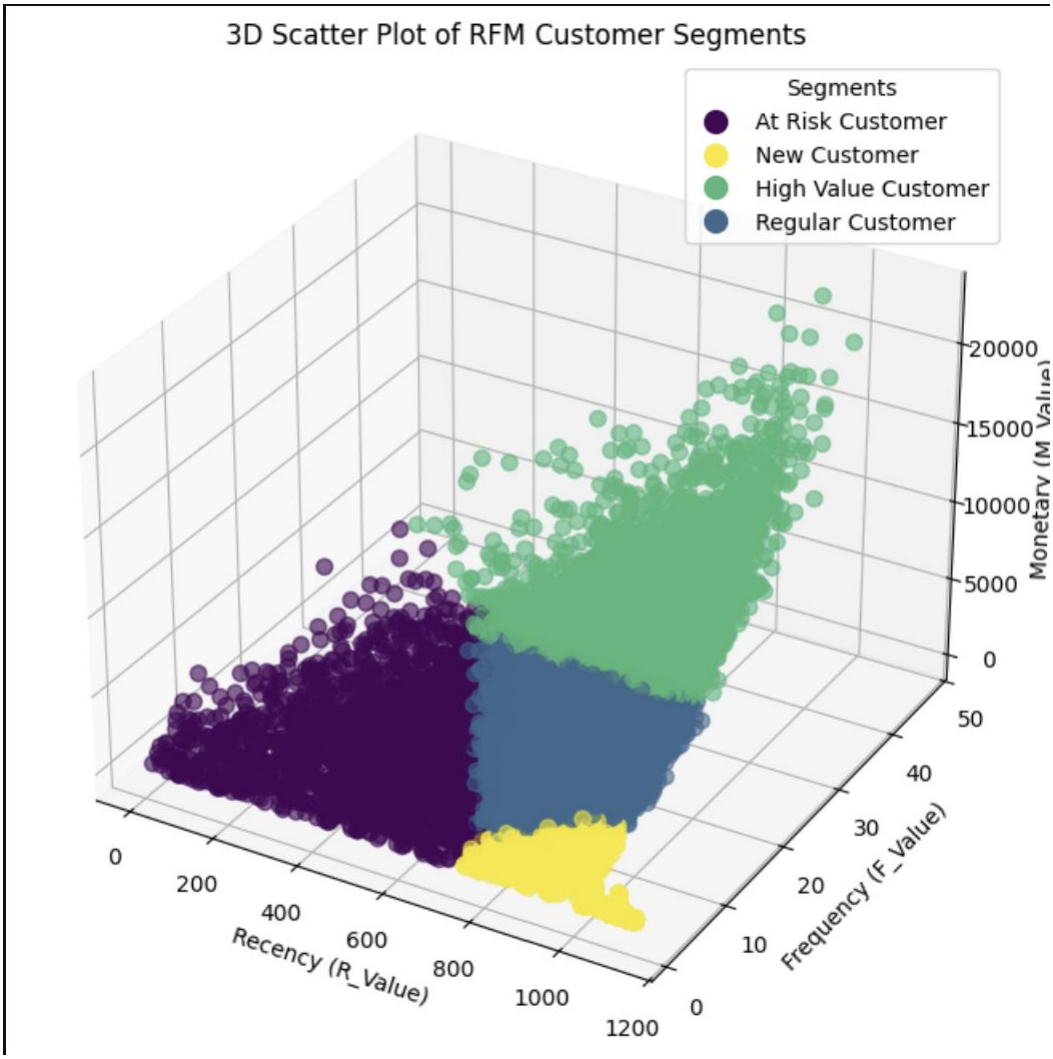


Figure 15 3D Scatterplot Visualization of Clustering Result

APPENDIX 16: Table 1

Table 1 Model Performances Summary

	Classification Model	Interventions	Features	Training Accuracy Score	Test Accuracy Score	Recall Score	Precision Score	F1 Score
0	KNeighborsClassifier	null	45	98.100000	97.680000	4.820000	60.290000	8.930000
1	DecisionTreeClassifier	null	45	98.480000	98.270000	44.120000	71.430000	54.550000
2	RandomForestClassifier	null	45	97.780000	97.650000	0.000000	100.000000	0.000000
3	LogisticRegression	null	45	97.890000	97.760000	18.820000	57.760000	28.390000
4	ExtraTreesClassifier	null	45	97.780000	97.650000	0.000000	100.000000	0.000000
5	XGBClassifier	null	45	98.740000	98.470000	45.060000	82.010000	58.160000
6	XGBClassifier	Hyperparameter Tuned XGBoost	45	100.000000	99.390000	78.590000	94.750000	85.920000
7	Neural Network	null	45	97.775853	97.645688	97.645690	95.346808	96.482557
8	KNeighborsClassifier	PCA	20	98.060000	97.630000	3.650000	45.590000	6.750000
9	DecisionTreeClassifier	PCA	20	98.470000	98.020000	35.880000	64.080000	46.000000
10	RandomForestClassifier	PCA	20	98.160000	97.860000	10.590000	88.240000	18.910000
11	LogisticRegression	PCA	20	97.780000	97.650000	0.000000	100.000000	0.000000
12	ExtraTreesClassifier	PCA	20	97.780000	97.650000	0.000000	100.000000	0.000000
13	XGBClassifier	PCA	20	98.580000	98.170000	36.240000	72.470000	48.310000
14	Neural Network	PCA	20	97.775853	97.645688	97.645690	95.346808	96.482557
15	KNeighborsClassifier	Feature Importance	20	97.980000	97.560000	2.350000	28.170000	4.340000
16	DecisionTreeClassifier	Feature Importance	20	97.980000	97.700000	7.060000	60.000000	12.630000
17	RandomForestClassifier	Feature Importance	20	97.780000	97.650000	0.000000	100.000000	0.000000
18	LogisticRegression	Feature Importance	20	97.780000	97.650000	0.000000	100.000000	0.000000
19	ExtraTreesClassifier	Feature Importance	20	97.780000	97.650000	0.000000	100.000000	0.000000
20	XGBClassifier	Feature Importance	20	97.800000	97.650000	0.350000	100.000000	0.700000
21	Neural Network	Feature Importance	20	97.775853	97.645688	97.645690	97.701118	96.482557
22	Neural Network	Oversampling + auto	45	99.568945	97.991914	97.750942	97.801524	96.639202
23	Neural Network	Feature Importance & Oversampling + auto	20	98.298800	96.233106	96.233104	96.662581	96.439608
24	Neural Network (Manually Tuned)	Manual Hyperparameter Tuning + Class Weights	20	60.774851	61.275756	61.275759	95.872884	74.089620
25	Neural Network	Feature Importance, Oversampling Auto, Infrast...	20	97.749543	97.750944	97.750942	97.801524	96.639202

APPENDIX 17: Table 2

Table 2 Neural Network Summary

Layer (type)	Output Shape	Param
dense (Dense)	(None, 1024)	47,104
dense_1 (Dense)	(None, 512)	524,800
dense_2 (Dense)	(None, 256)	131,328
dense_3 (Dense)	(None, 128)	32,896
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 32)	2,080
dense_6 (Dense)	(None, 16)	528
dense_7 (Dense)	(None, 8)	136
dense_8 (Dense)	(None, 4)	36
dense_9 (Dense)	(None, 2)	10
dense_10 (Dense)	(None, 1)	3

APPENDIX 18: Figure 16

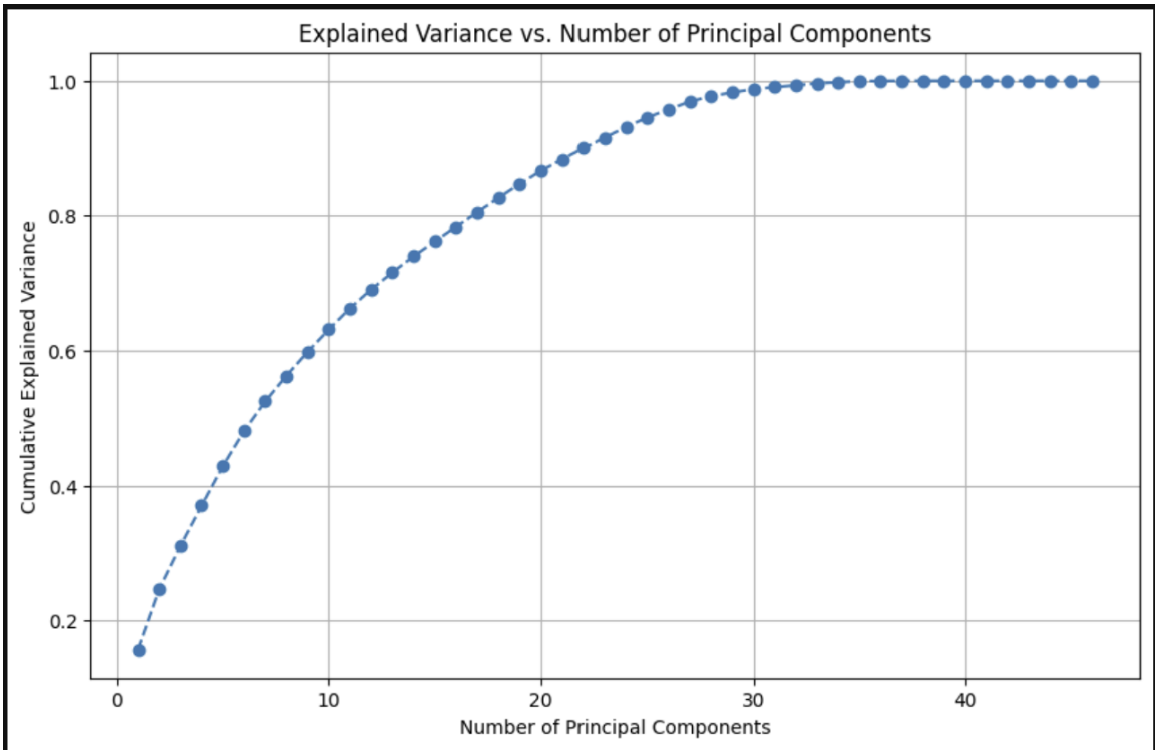


Figure 16 Explained Variance vs Number of Principal Components

APPENDIX 19: Table 3

Table 3 Final Neural Network Infrastructure Summary

Layer (type)	Output Shape	Param #
dense_59 (Dense)	(None, 1024)	21,504
batch_normalization_7 (BatchNormalization)	(None, 1024)	4,096
dropout_2 (Dropout)	(None, 1024)	0
dense_60 (Dense)	(None, 512)	524,800
batch_normalization_8 (BatchNormalization)	(None, 512)	2,048
dropout_3 (Dropout)	(None, 512)	0
dense_61 (Dense)	(None, 256)	131,328
batch_normalization_9 (BatchNormalization)	(None, 256)	1,024
dropout_4 (Dropout)	(None, 256)	0
dense_62 (Dense)	(None, 128)	32,896
batch_normalization_10 (BatchNormalization)	(None, 128)	512
dropout_5 (Dropout)	(None, 128)	0
dense_63 (Dense)	(None, 64)	8,256
batch_normalization_11 (BatchNormalization)	(None, 64)	256
dropout_6 (Dropout)	(None, 64)	0
dense_64 (Dense)	(None, 32)	2,080
batch_normalization_12 (BatchNormalization)	(None, 32)	128
dropout_7 (Dropout)	(None, 32)	0
dense_65 (Dense)	(None, 16)	528
batch_normalization_13 (BatchNormalization)	(None, 16)	64
dropout_8 (Dropout)	(None, 16)	0
dense_66 (Dense)	(None, 1)	17

APPENDIX 20: Figure 17

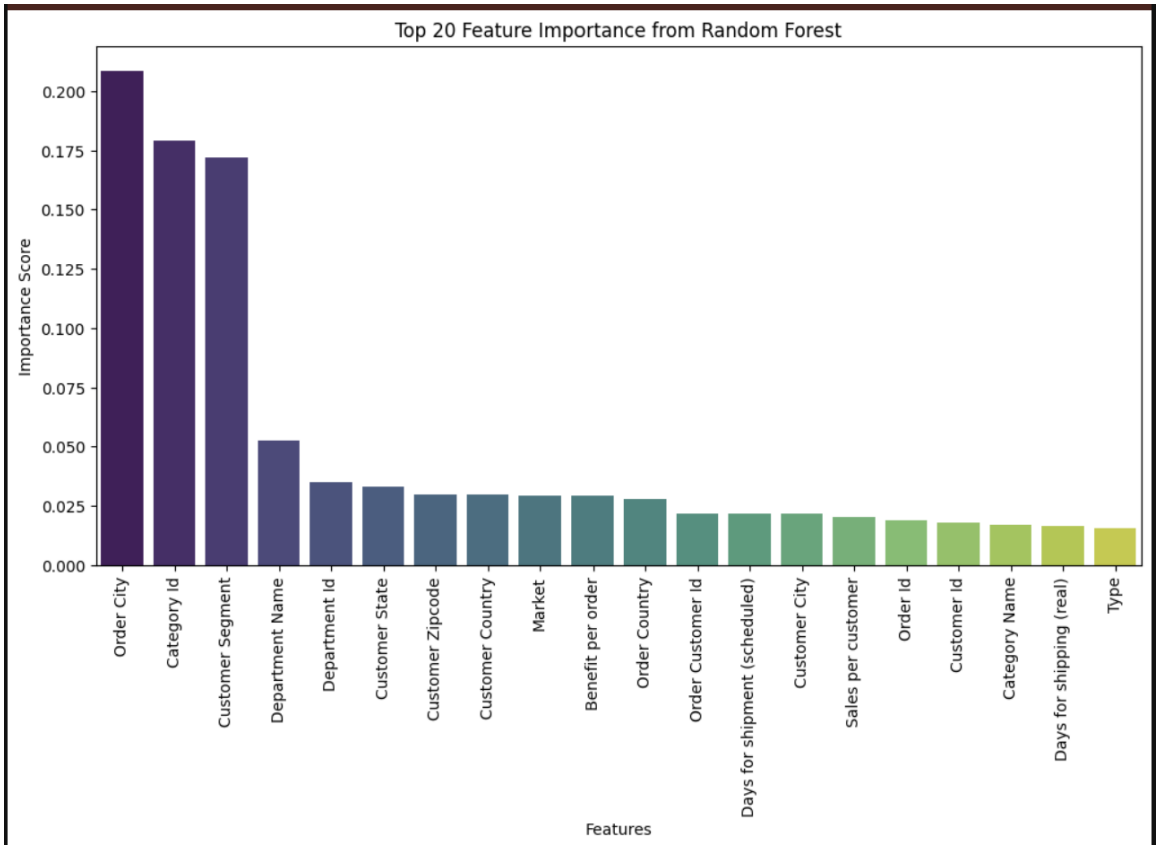


Figure 17 Top 20 Important Features

APPENDIX 21: Figure 18

Figure 18 Project Workflow